Natural Language Processing

Lecture notes from my graduate level Natural Language Processing class taught by professor James H. Martin at the University of Colorado Boulder. This course covers a range of topics in natural language processing including, but not limited to: probabilistic language modeling, text classification, vector semantics, neural networks, large language models, and transformer architecture. The textbook used is the 3rd Edition of *Speech and Language Processing* by Dan Jurafsky and James H. Martin. The draft of the 3rd edition can be found here: https://web.stanford.edu/~jurafsky/slp3/

Disclaimer: This document are notes taken by the author (me) a student in graduate school as way to internalize the material I am learning in this class. As such, these notes will inevitably contain mistakes such as errors, typos, etc., as well as personal anecdotes, quips, opinions, and biases. Do what you will with that information, and as a professor once said to me "It's better not to think about it too much".

Fall Semester 2026

University of Colorado Boulder

Contents

1 Large Language Models **1**
1.1 What is NLP? . . . 1
1.2 The Three Pillars of modern NLP . . . 1
1.3 The entire brief history of speech and language processing (I guess) . . . 1
1.3.1 Foundations of NLP . . . 1
1.3.2 The Symbolic Turn . . . 2
1.3.3 Oh my gosh! Empiricism is back! . . . 2
1.3.4 Just Chat it. . . . 2
1.4 Linguistic Structure . . . 2
1.4.1 Syntactic structure . . . 2
1.4.2 Co-reference . . . 2

2 Words and Tokens **3**
2.1 Morphology . . . 3
2.1.1 Language classification . . . 4
2.1.2 Morphological typology . . . 4
2.2 So, what is a word? . . . 4
2.2.1 Vocabulary . . . 4
2.2.2 Corpora . . . 4
2.3 Heap's Law . . . 5
2.4 Text Normalization and Tokenization . . . 5
2.4.1 Issues in normalization . . . 5
2.4.2 Sentence segmentation . . . 6
2.4.3 Multilingual issues in normalization . . . 6
2.4.4 Tokenization . . . 6
2.5 Byte-Pair Encoding (BPE) . . . 6
2.5.1 Multilingual issues with tokenization . . . 7
2.5.2 Does vocab size matter? . . . 8

3 N-Gram language models **8**
3.1 What is a language model? . . . 8
3.1.1 Word prediction . . . 8
3.1.2 The Markov assumption . . . 8
3.2 Smoothing . . . 9
3.2.1 Additive smoothing . . . 9
3.3 Evaluating language models . . . 10
3.4 Log probabilities . . . 10
3.4.1 Perplexity and entropy . . . 10

4 Logistic regression and text classification **11**
4.1 Classification . . . 11
4.1.1 Text classification . . . 11
4.2 Logistic regression . . . 11
4.2.1 Scoring . . . 11
4.2.2 The sigmoid function . . . 12
4.2.3 From probabilities to odds . . . 12

4.2.4 Now from odds to log odds . . . 12
4.3 Loss functions . . . 13
4.3.1 Cross-entropy loss . . . 13
4.4 Gradient descent . . . 14
4.4.1 Stochastic gradient descent . . . 15
4.5 Batch training . . . 15
4.6 Okay, now apply it to the multinomial case . . . 15
4.6.1 Softmax . . . 15
4.6.2 Cross entropy loss in the multinomial sense . . . 16
4.6.3 Gradient descent . . . 16

5 Embeddings **16**
5.1 Lexical semantics . . . 16
5.1.1 Lemmas and senses . . . 17
5.1.2 Synonyms . . . 17
5.1.3 Word similarity . . . 17
5.1.4 Word relatedness . . . 17
5.1.5 Connotation . . . 17
5.2 Vector semantics . . . 18
5.2.1 Sparse . . . 18
5.2.2 Dense . . . 18
5.3 Simple count-based embeddings (Sparse) . . . 18
5.4 Cosine for measuring similarity . . . 19
5.5 Word2Vec . . . 19
5.5.1 Pros of word2vec vs neural network . . . 19
5.5.2 How skip-gram works . . . 20
5.5.3 Skip-gram embeddings . . . 20

6 Neural Networks **21**
6.1 Units . . . 21
6.2 XOR problem . . . 22
6.2.1 Neural networks! . . . 22
6.3 Feed-forward neural networks . . . 22
6.4 Feed-forward networks for NLP classification . . . 23
6.4.1 How it works in practice . . . 23
6.5 Training NNs . . . 24
6.6 Loss function . . . 24
6.7 Gradient . . . 24
6.8 Computation graphs . . . 24
6.8.1 Backwards pass . . . 25
6.9 Recurrent NNs . . . 25

7 Transformers **25**
7.1 Attention . . . 26
7.1.1 Word embeddings vs contextual embeddings . . . 26
7.1.2 Self-attention . . . 26
7.1.3 Attention head . . . 26
7.1.4 Multi-head attention . . . 27

7.2 Transformer blocks 28
7.2.1 Feed-forward layer 28
7.2.2 Layer norm 28
7.2.3 All together now 29

1 Large Language Models

A large language model is a computational system that can predict the next word from previous words. Given a context of previous words a language model assigns probabilities what the next word will be.

For example, when given the sentence "so long and thanks for ...", the next word that comes after that is most likely to be guess "all" or "the" (my mind immediately went to Fall Out Boy's song *Thnks fr th Mmrs*, "thanks for the memories"). Unlikely words would probably be "diagonalize" or "bipartite".

The language model is a large neural network that takes in context and returns a probability of the next word. We can also use this prediction to **generate** text by sampling from a distribution (we just generate a random number and choose a word based on the probability assigned to it).

1.1 What is NLP?

Any language tech involved with generating, manipulating, etc. of text.

- e.g., spellcheck, dictation, translation, text-to-speech

Computational Linguistics: Modeling human language using computers so we can understand how languages work

- linked to psychology, philosophy, and cognitive science.

1.2 The Three Pillars of modern NLP

1. **Probabilistic Language Models**: systems that assign probability distributions over possible next words.
2. **Neural Networks**: the fundamental computational mechanism. Modeled after human brain neurons turning into deep architectures optimized via gradient descent.
3. **Embeddings**: vector-based representations where meaning is mapped as a point in high-dimensional continuous space.

1.3 The entire brief history of speech and language processing (I guess)

The central ideas of NLP lie way back in the 1940s to the early 60s.

1.3.1 Foundations of NLP

- Shannon (1951) developed ideas about word prediction
- Labs at Cornell and Stanford were applying small NNs to computational tasks.
- In 1957 psychologist Osgood proposed that a meaning of a word could be modeled by a point in space
  - Linguists proposed that a word's meaning is related to its context.

1.3.2 The Symbolic Turn

- Modeling language in terms of structure appeared.
- Chomsky argued against statistical modeling and neural models of a language which led to a shift in computer science, AI, and linguistic and cognitive sciences.
- Symbolic approaches become more popular.

1.3.3 Oh my gosh! Empiricism is back!

The field moved back to statistical and neural models (thanks, Chomsky) lasting from 1975 and 1988 to 2017.

- Speech recognition from IBM became a thing.
  - N-gram models and algorithms were developed for this purpose
- Improved algorithms for NN training based on error backpropagation became popular.
  - Led to connectionist/parallel distributed processing
- First neural language models was proposed in 2003.
- Transformer architecture becomes popular for general-purpose neural architecture for various tasks

1.3.4 Just Chat it.

Before prompting, you had to label training data to train a classifier to assign a label to data. Now you can just prmopt! What is prompting? Given an algorithm that can predict words, just type in a question and append something like "the answer is" and it would predict the answer!

After this, NLP became about generation rather than algorithms that can parse/interpret text. This paved the way for **generative AI** which focuses on generation of text, images, video, speech, etc.

1.4 Linguistic Structure

Understanding human language and their properties are important for studying speech and language processing.

1.4.1 Syntactic structure

Abstract grammar relationships between words or phrases. For example, verbs have subjects and direct objects

She announced this in January

Detecting these relationships is what **parsing** can do.

1.4.2 Co-reference

When a text talks about a person, it can refer to them in many ways.

Victoria, the CFO of J.P Morgan, saw her pay jump from $2.4 million, as the 32-year old became the company's president

"Victoria" is the "CFO" and the "32-year old". They refer to the same entity "Victoria", just with different ways of saying it.

2 Words and Tokens

- drives LLMS these days
- fundamental bridge from linguistic forms to underlying conceptual representations
- **type**: entries in vocab/lexicon
- how do we manage words/tokens?

2.1 Morphology

- the study of the ways words are built up from smaller units called morphemes.
- **Two classes of morphemes**:
  - **stems**: the core meaning-bearing unit
  - **affixes**: bits and pieces that attach to stems but do not change the core meaning of the word example: "star_s" with the core being "star" and the "-s" being the plural affix we attach.

Inflectional morphology

The combination of stems and affixes where the resulting word ...

- has the same word class
- serves a grammatical purpose that is
  - different from the original
  - transparently related to the original
    - "walk" + "ing" = walking
    - "cat" + "s" = cats

Derivational morphology

Messy stuff that nobody ever "taught" when learning English

- wide variety of affixes
  - -al, -tion, -ee, -er, -ant, -ness
  - anti-, un-, non-
- quasi-systematic rules
- irregular meaning change

- changes of word class

2.1.1 Language classification

- **Genetically**: groups based on family history
- **Areal**: geographical occurences
- **Typological**: shares grammar, morphology, phonology, syntax, etc.

2.1.2 Morphological typology

Groups languages based on aspects of morphology

Isolating vs Synthetic

- **Isolating**: one morpheme per word
- **Synthetic**: lots of morphemes per word

Agglutinating vs fusional

- **Agglutinating**: form words by concatenating stems with affixes to form words (with prefixes, suffixes, infixes)
- **Fusional**: allow morphemes to change form upon inflection; harder to isolate them

Computational morphology

Morphological analysis takes surface form and returns the stem with a set of morphological features rather than segmenting the surface form.

2.2 So, what is a word?

2.2.1 Vocabulary

- a vocabulary is the set of words a system has access to
- the minimal units of processing in an input that has some semblance of meaning attached to it.

What should such a vocabulary (or lexicon) consist of?

- all the words in a language?
- all the words in some comprehensive dictionary?
- some subset?

2.2.2 Corpora

Words don't just appear out of nowhere. Corpora are the representative texts that are used either generically for diverse applications or designed for a specific domain or application.

1. Collect a big bunch of text.

2. Figure out the words that appear in it.
3. That's your vocabulary

Dimensions

- **Language**: dialects, creoles, pidgins, etc.
- **Provenance**: how was it obtained? Consent? Legal?
- **Genre**: Newswire, fiction, non-fiction, scientific articles, Wikipedia
- **Demographics**: Writer's age, gender, race, socioeconomic status, etc.
- **Medium**: Spoken, written, captioned, video
- **Writing system**: Over 350 systems in use

How many words are there?

N = number of mentions V = vocabulary = set types ($|V|$ is the size of the vocab)

2.3 Heap's Law

$$|V| = kN^{\beta}$$

Where k and β are free variables. Usually k between 10 and 100 and β between 0.6 and 0.7.

- Size of the vocabulary grows somewhat faster that the square root of the corpus size
- The rate of the vocabulary growth slows as corpus grows, but never flattens out.

2.4 Text Normalization and Tokenization

This is essentially the question of "how do we process a corpora and take it from messy to clean and something that we can analyze?"

Normalizing text can look like

- Segmenting running text into sentences
- Tokenizing sentences into words
- Normalizing word formats
- Recognizing different scripts/languages

2.4.1 Issues in normalization

- Hong Kong → one word? two?
- m.p.g → is this even a word?
- IRA, I.R.A → expand? keep out?

2.4.2 Sentence segmentation

Tokenization, morphology, and further syntactic processing first requires sentence segmentation

- Easy in English. Punctuation is used to mark sentence boundaries.
  - "!", "?" are relatively unambiguous.
  - Periods can be. Like "Dr.", "I.R.A", etc.

2.4.3 Multilingual issues in normalization

English is not the only language out there, and there are several problems that need to be faced.

- Code switching — "Chinglish", "Spanglish", etc.
- Writing systems — Latin, Chinese, Arabic
- Complex morphology — words built with two or more morphemes
- Sentence segmentation — Chinese, Thai, Japanese do not use spaces to separate words

2.4.4 Tokenization

Tokenization is the process of segmenting text into tokens

Given a training corpus, subword tokenization identify the base word and subwords in a vocabulary empirically.

Tokens are essentially a (crude) approximation to morphemes. They are minimal units to which we can associate meaning in a model. In worst case scenarios, a word can be decomposed into characters.

For tokenization to work you need two things:

- A learner that takes the raw training corpus and makes a vocabulary of a specific size
- A tokenizer (encoder) that takes input text and tokenizes it according to the vocabulary

2.5 Byte-Pair Encoding (BPE)

Byte Pair Encoding works by merging neighboring tokens k times to create longer and longer tokens.

The core of the algorithm is

- Choose the most frequently occurring adjacent pair of vocabulary items in the training corpus
- Add the newly merged symbol to the vocabulary
- Replace every occurrence of the adjacent pair with said merged symbol
- Redo the counts
- Repeat k times

BPE Example

Let's see how BPE really works with this sentence that will never exist beyond this example in the English language (see: infinite monkeys with infinite typewriters)

`set_new_new_renew_reset_renew`

First, we can break up the corpus into words, with their counts (including whitespace!).

corpus	**vocabulary**
2 _ n e w	_, e, n, r, s, t, w
2 _ r e n e w	
1 s e t	
1 _ r e s e t	

The BPE will iterate and counts all pairs of adjacent symbols. The most frequent in this corpus is `ne` which occurs in "new" and "renew". We merge those symbols and recount

corpus	**vocabulary**
2 _ne w	_, e, n, r, s, t, w, ne
2 _r e ne w	
1 s e t	
1 _ r e s e t	

Now the most frequent pair is `ne` + `w`. So we merge and recount

corpus	**vocabulary**
2 _new	_, e, n, r, s, t, w, ne, new
2 _r e new	
1 s e t	
1 _ r e s e t	

Merge 3 & 4 are `_r` merge and `_re` merge respectively.

corpus	**vocabulary**
2 _new	_, e, n, r, s, t, w, ne, new, _r, _re
2 _re new	
1 s e t	
1 _re s e t	

If we do that enough times merges 5–8 look like this

merge	**vocabulary**
(_, new)	_, e, n, r, s, t, w, ne, new, _r, _re, _new
(_re, new)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew
(s, e)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew, se
(se, t)	_, e, n, r, s, t, w, ne, new, _r, _re, _new, _renew, se, set

2.5.1 Multilingual issues with tokenization

Most models are assumed to be monolingual. Multilingual models were built for the use of translation, retrieval, etc. While most frontier models have "multilingual" capabilities, they weren't built that way. It just so happens that when you treat the Internet akin to a kid in a candy store, the models "learn" other languages just by happenstance.

2.5.2 Does vocab size matter?

It does! Smaller vocabularies have lower memory requirements, but they might miss more tokens. But smaller vocabularies lead to longer inputs, because the smaller words lead to more tokens (commercial LLMs charge by token).

3 N-Gram language models

3.1 What is a language model?

A language model is a machine learning model that predicts words. More elegantly, it assigns probability to each possible next word, or gives a probability distribution of upcoming words that could may or may not be possible.

3.1.1 Word prediction

Given a certain word, we can formalize this prediction of "given this word, what is the probability of this word occurring next?" as

$$P(w_n|w_{1:n-1})$$

With the definition of conditional probabilities

$$P(B|A) = \frac{P(A,B)}{P(A)}$$
$$P(A,B) = P(A)P(B|A)$$

...as well as the **Chain Rule** we can think of word prediction as a sequence of a joint event

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_2,w_1)\dots P(w_n|w_1\dots w_{1:n-1}) \\ &= \prod_{i=1}^{n} P(w_i|w_{1:i-1}) \end{aligned}$$

Problem: The Chain Rule can suggest that the probability of a given word consists of the joint probability of multiplying a number of conditional probabilities, but language is creative, and a sentence can really just appear out of nowhere. (There are definitely some sentences that I have had the horror of experiencing that I don't ever want to experience again).

3.1.2 The Markov assumption

Ok, so we can't count the probability of *all* the previous words, but what about narrowing that scope? What about if we predict a word based on the last word? Last two? Three?

The bigram model

The bigram model gives us a probability of the next word given **just** the previous word $P(w_n|w_{n-1})$.

The assumption that the probability of a word depends just on the previous word is called a **Markov** assumption.

General cases

Bigram version

$$P(w_i|w_{i:1}) \approx P(w_i|w_{i:i-1})$$

Trigram version

$$P(w_i|w_{i:1}) \approx P(w_i|w_{i-2:i-1})$$

General case

$$P(w_i|w_{i:1}) \approx P(w_i|w_{i-(N-1):i-1})$$

Combining this with the Markov assumption and we get

$$P(w_{1:n}) \approx \prod_{i=1}^{n} P(w_i|w_{i-(N-1):i-1})$$

3.2 Smoothing

Language is such an interesting thing. Because of this, when training a language model, sometimes a probability of two words being together like "sincere politician" may appear in the test set, but not the training set! This would make the probability that bigram 0.

To deal with this we can use smoothing which is just — as Professor Martin put it — stealing too much from the rich and giving it to the poor.

3.2.1 Additive smoothing

Add a little bit to all the counts and the zeroes magically go away!

For unigrams it would look like

$$P_{+1}w = \frac{\text{Count}(w) + 1}{M + V}$$

For bigrams

$$P_{+1}(w_n|w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n) + 1}{\text{Count}(w_{n-1}) + V}$$

3.3 Evaluating language models

Best way to evaluate a model is to embed it into an application and measure its performance. This is called **extrinsic evaluation**. It's really the only way to know how well a language model will do with a given task.However, this is very expensive. So to cut costs we can do **intrinsic evaluation** which is testing the language model outside the application.

To do this, we have to have three datasets: **training set**, **dev set**, **test set**.

The **training set** is the dataset we use to train the model on. Like a study guide for an exam.

The **test set** is the exam itself. It's what is used to evaluate the model. This dataset is distinct from the training set because we don't want the model to memorize the exam answers. We want it to study, and then test its knowledge with data it's never seen before.

The **development set** is the interlude test set which we can evaluate the performance of our model before we actually use the test set. It's like a "mock-exam" per se.

3.4 Log probabilities

Using raw probabilities are related to the length of sequences and number of sentences in the test set. The more probabilities we multiply together, the smaller the number becomes.

To deal with this, we use the log space to normalize and get results that are not as small.

$$H(T) = -\frac{1}{N}\sum_{i=1}^{N} \log P(w_i|w_{i:i-1})$$

...where $H(T)$ is the entropy.

3.4.1 Perplexity and entropy

Perplexity is traditionally THE intrinsic measure used to evaluate large language models (like the AI model Perplexity!), especially with machine translation and speech recognition.

The perplexity of a model is 2 raised to the entropy!

$$PP(T) = 2^{H(T)}$$

Perplexity is roughly the average number of choices that a model would have to make in guessing the next item in a sequence.

- if it is 100% certain then the perplexity is 1 (entropy of 0 since log(1)=0), because there is no choice.
- if it has no information and there are M choices, the perplexity is M which makes sense! it's just guessing randomly.

4 Logistic regression and text classification

4.1 Classification

Classification is the core of NLP. It's the task of examining an experience, event, entity and assigning it to a known class or category.

By placing known things into categories, we can make good reasons to what unknown items belong in certain categories based on what we know about current members in a class.

Some machine learning classifiers include:

- Speech recognition
- Image classification
- Intrusion detection

4.1.1 Text classification

The task of assigning a label or category to a text/document. Some examples are categorizing reviews, spam detection, authorship, etc.

With language classification, each word can be thought of as a class, so **predicting the next word** can be thought of as classifying the context-thus-far into a class for each next word!

4.2 Logistic regression

The goal of classification is to take a single input, find some useful features from the input, and then use that to predict an unknown case.

With logistic regression we are trying to classify inputs into either a positive case or negative case. Logistic regression models estimate the conditional probability of a class assignment given a text.

$$P(c|t)$$

4.2.1 Scoring

Given features and weights, we can assign a score to each class for a given function.

$$z = b + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

Where b is a bias and w are the weights assigned to each class x.

We can think of these as vectors if we want to ~~begrudgingly~~ use linear algebra

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \cdots \\ w_n \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix}$$

$$z = w^\top x + b$$

We can eliminate complexity by incorporating the bias as w_0 with $x_0 = 1$

$$\begin{aligned} w_0 &= b \\ x_0 &= 1 \\ z &= w_0 x_0 + w_1 x_1 + \cdots + w_n x_n \\ z &= w^\top + x \end{aligned}$$

4.2.2 The sigmoid function

Logistic regression shines with how we can convert the score we just calculated into a probability. To do this cleanly, we can squash the scores between 0 and 1 using the logistic sigmoid function.

$$\begin{aligned} P(y = 1|x) &= \sigma(w^\top x) \\ &= \frac{1}{1 + e^{-w^\top x}} \\ P(y = 0|x) &= 1 - \sigma(w^\top x) \end{aligned}$$

4.2.3 From probabilities to odds

Now that we have probabilities calculated via the sigmoid function $p = \frac{1}{1+e^{-z}}$ let's convert it to odds.

$$\text{odds} = \frac{p}{1-p}$$

Probabilities range from 0 to 1, but with odds, they range from 0 to infinity.

4.2.4 Now from odds to log odds

Everything is just logs . . . But they are useful!

$$ln(\frac{p}{1-p})$$

By taking the log odds, we can get some nice symmetry with opposing odds

0.95	19 to 1	2.994
0.5	1 to 19	-2.994

Logit

The "log of the odds" is often referred to as the **logit**

- z scores from $w^\top x$ computation are logits
- now how do we take a logit and get a probability?

$$
\begin{aligned}
z &= ln(\frac{p}{1-p}) \\
e^z &= \frac{p}{1-p} \\
e^z(1-p) &= p \\
e^z - pe^z &= p \\
e^z &= p + pe^z \\
e^z &= p(1+e^z) \\
p &= \frac{1}{1+e^{-z}}
\end{aligned}
$$

Summary

- The logit is the log of odds of some event. The odds being the ratio of the probability of an event happening or not.
- The raw scores from a linear classifier are the logits.
- Now, how do we come up with a learning scheme to acquire the proper weights so that our scores represent logits appropriate for a specific task?

4.3 Loss functions

Now that we have the probability of our classifier assigning a specific class for both the binary case and all other non-binary cases (*I did that on purpose*), we want to ask the question: how close are our answers to the correct answers?

That's what **loss functions** are for!

4.3.1 Cross-entropy loss

$$p(y|x) = \hat{y}^y(1-\hat{y})^{1-y}$$

where y is the correct answer and $\hat{y}$ is our answer.

With our loss function we want **bad performance** to have high loss, and **good performance** to have low loss. So, we take the negative of the log probability assigned to the correct answer as the loss.

Log of loss

Now, what do we do to every formula so far? We take the log of it!

$$\begin{aligned} p(y|x) &= \hat{y}^y(1-\hat{y})^{1-y} \\ \log(p(y|x)) &= \log[\hat{y}^y(1-\hat{y})^{1-y}] \\ &= y\log\hat{y} + (1-y)\log(1-\hat{y}) \end{aligned}$$

$$L_{CE}(w,b) = -[y\log\sigma(w\cdot x+b) + (1-y)\log(1-\sigma(w\cdot x+b))]$$

Taking the log of the loss will take the range from 0 to 1 and make the range from -infinity to 0.

Now taking the negative of that will take the range from -infinity to 0 and make the range 0 to infinity.

So with a low probability we get a high value, and a high probability a low value. This satisfies our requirement for bad performance = high loss, good performance = low loss.

4.4 Gradient descent

Now that we can find how much loss there is, we need a way to minimize the loss by updating the weights.

$$\begin{aligned} Cost(w,b) &= \frac{1}{m}\sum_{i=1}^{m} L_{CE}(\hat{y}^i, y^i) \\ &= -\frac{1}{m}\sum_{i=1}^{m} y^i\log\sigma(w\cdot x^i+b) + (1-y^i)\log(1-\sigma(w\cdot x^i+b)) \end{aligned}$$

Gradient descent works by figuring out in which direction the function's slope is rising steeply, and moving it in the opposite direction. A good example that the textbook gives is if you're hiking down a canyon, you would first look around to find which direction the ground is sloping the steepest and move opposite that direction.

The magnitude and direction of change that we should make to each of the weights is dictated by the derivative of the loss function. A higher (faster) learning rate means that we should move the weight more on each step.

$$w_{t+1} = w_t - \mu\Delta$$

This is just for one weight. In actual application, we have multiple weights for multiple classes. To make this happen we use the partial derivatives of the loss with respect to (wrt) the weights. This is called the gradient.

$$\frac{\partial L_{CE}(w,b)}{\partial w_j} = [\sigma(w\cdot x+b) - y]x_j$$

Essentially what this is doing is taking the difference of the computed answer $(w \cdot x + b)$ and the right answer y and multiplying it by the value of the input feature for the weight being adjusted x_j.

This can also be written as

$$(\hat{y} - y)x_j$$

or

$$-(y - \hat{y})x_j$$

4.4.1 Stochastic gradient descent

The Stochastic Gradient Descent minimizes the loss function by computing the gradient after each training example, nudging θ in the right direction (the opposite direction of the gradient). It's called **stochastic** because it chooses a single random example at a time (which I didn't know that "stochastic" meant random, so I learned something new today!)

4.5 Batch training

Computing the gradient over the entire dataset. This offers a really good estimate of which direction to move the weights with the cost of not spending a lot of time on individual examples in the data.

Mini-batch training is instead of computing the gradient over the entire dataset, we compute the gradient over an m sized batch in the dataset. It's a good compromise from only one gradient update and updating the gradient after *every* single iteration.

4.6 Okay, now apply it to the multinomial case

In most cases, we want to evaluate between more than just the binary cases. The easiest way to do this is assign each class it's own set of feature weights to assess the score for each class. The resulting score can be normalized to produce a probability distribution over all the classes.

4.6.1 Softmax

To extend our current logistic classifier to the multinomial sense, we need a generalized version of the sigmoid called the softmax function to compute $p(y_k = 1|x)$.

The softmax takes a vector of arbitrary attributes and maps them to a probability distribution with each value in the range of $[0, 1]$ and all the values summing to 1.

$$P(y = c|x; w) = \frac{e_c^z}{\sum_{j=1}^{k} e_j^z}$$

e_c^z is the raw score of a single class c which exapnds to $z = \sigma(w_c \cdot x + b_c)$

$$p(y = c|x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{j=1}^{k} e^{w_j \cdot x + b_j}}$$

Given object x the softmax function gives us the probability distribution over the classes.

4.6.2 Cross entropy loss in the multinomial sense

We can take the formula we used before for the binomial case and apply it for k amount of classes

$$\begin{aligned} L_{CE}(\hat{y}, y) &= -\sum_{k} = 1^K y_k \log(\hat{y}_k) \\ &= -\log(\hat{y}_c) \\ &= -\log \hat{p}(y_c = 1|x) \\ &= -\log \frac{\exp(w_c \cdot x + b_c)}{\sum_{j=1}^{K} \exp(w_j \cdot x + b_j)} \end{aligned}$$

4.6.3 Gradient descent

For the gradient descent of a multinomial case the partial derivative turns out to just be the difference between the true value for the class k (either a 1 or 0) and the probability the classifier outputs for class k weighted by the value of the input x_i for the ith element

$$\begin{aligned} \partial L_{CE} &= -(y_k - \hat{y}_k) x_i \\ &= -(y_k p(y_k = 1|x)) x_i \\ &= -\Big(y_k \frac{\exp(w_k \cdot x + b_k)}{\sum_{j=1}^{K} \exp(w_j \cdot x + b_j)}\Big) x_i \end{aligned}$$

This is just saying if we have k weights we will end up with k derivatives as a vector which is the gradient.

5 Embeddings

When dealing with a lot of words that can have similar meanings, **context** is important to consider when figuring out how words are related.

Words that occur in similar contexts tend to have similar meanings.

Ok, so how do we represent this?

Embeddings (insert title reference; we can all go home now). Embeddings are vector representations of the meaning of words learned directly from word distributions in texts.

5.1 Lexical semantics

How should we represent the meaning of a word?

5.1.1 Lemmas and senses

aka the **citation form**. like "pepper" or "mouse" (in this case "mouse" can be a lemma to "mice").

Another example is "sing", can be a lemma for "sang" or "sung".

Each lemma can have multiple meanings. Each of these meanings is called a **word sense**. This can make interpretation difficult.

5.1.2 Synonyms

Another way that words can relate is through **synonyms**. Words are synonyms if both their senses are nearly identical to each other.

couch/sofa, vomit/throw up, filbert/hazelnut, car/automobile

A more formal definition is "two words are synonymous if they are suitable for one another in any sentence without changing the *truth conditions* of the sentence".

Principle of contrast — difference in form is always associated with a difference in meaning.

This is used to describe how approximately a word is synonymous with another.

5.1.3 Word similarity

While words don't have many synonyms, words do have a LOT similar words. Take for example "cat" and "dog". They mean completely different things, but they are similar words.

Shifting towards similarity is important because it helps with tasks such as paraphrasing, summarizing, etc.

5.1.4 Word relatedness

Words can be related in many ways even if they aren't similar at all. The example the textbook gives is "coffee" and "cup". These words aren't similar at all, but they appear together in sentences all the time.

- Same "conceptual frame": buy, sell, product, price, merchant, goods, etc
- Part-whole: wheel, fender, roof, axle, tire, engine
- Functional relationships: car/gasoline
- Taxonomically: candid: coyote, dog, wolf, fox, jackals

5.1.5 Connotation

The aspects of a word's meaning that are related to a writer or reader's emotions, sentiment, opinions, etc.

- Positive/negative emotional state: happy, sad
- +/- evaluation: great, horrible
- +/- frame: thrifty, stingy

5.2 Vector semantics

The standard way to represent a word meaning in NLP. This helps us model the many aspects of word meaning described in the section above.

So how does this work?

Define the meaning of a word by its distribution in language use (neighboring words or grammatical environments).

- The idea is that two words that occur in very similar distributions have similar meanings

5.2.1 Sparse

In this approach, words are represented as a function of the counts of words they co-occur with, typically at the document level.

The vector dimensionality is often the length of the vocabulary. With this approach, the vectors are sparse (contain lots of 0s)

5.2.2 Dense

1. Representations trained from larger corpora using semi-supervised learning based on co-occurrence statistics.
2. Mostly non-zero, floats. Dimensionality varies from hundreds to thousands.
3. Dimensionality is typically lower than vocab size.

5.3 Simple count-based embeddings (Sparse)

Cool, so how do we actually use them?

One way to compute word vector embeddings is by using a **co-occurrence matrix**.

A particular type of co-occurrence matrix, the **word-content matrix**, will represent how often each other word in vocabulary appears nearby.

Nearby is so ambiguous, so wtf do you mean by that??

The textbook uses the following example:

Let's say a context window that is "nearby" the target word could be 4 words to the left and right. Let's use the four words *cherry, strawberry, digital, information.* The words ± 4 for each of these could look like this

is traditionally followed by **cherry** pie, a traditional desert

often mixed, such as **strawberry** rhubarb pie. Apple pie

computer peripherals and personal **digital** assistants. These devices usually

a computer. This includes **information** available on the Internet

We can then take every occurrence of each word, count the context words around it, and get a word-context co-occurrence matrix where each row of the table is a vector representing a word.

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

A **vector space** is a collection of vectors and is characterized by its **dimension**.

We can think of this vector for a word as a point in $|V|$-dimensional space.

5.4 Cosine for measuring similarity

To measure how similar word v and word w are, we need a metric that takes two vectors and gives a measure of their similarity.

The **cosine** of the metric is the most commonly used metric. It is based on the dot product operator (aka the inner product)

$$\text{dot product}(v, w) = v \cdot w = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + \cdots + v_N w_N$$

The cosine similarity can be computed as

$$\text{cosine}(v, w) = \frac{v \cdot w}{|v||w|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

5.5 Word2Vec

Now we can get to **embeddings**. Embeddings are short, with the number of dimensions only ranging from 50-1000 rather than a large vocabulary size. All the vectors are **dense**: instead of having a lot of zeroes, we now have negative numbers!

Dense vectors tend to work out better than sparse vectors for NLP. Why? Many reasons such as

- Representing words as 300-dimensional dense vectors requires less weights to be learned.
- Smaller parameter space possibly helps with generalization and avoids overfitting.
- Captures synonymy better.

Word2vec works by instead of counting how often each context word c occurs near "apricot", we'll instead train a classifier on a binary prediction task that answers the question "Is word c likely to show up near *apricot*?

This employs a method called **self-supervision** where we can just run text as implicitly supervised training data and don't need to touch it!

5.5.1 Pros of word2vec vs neural network

- Simplifies the task by making it a binary classification and not word prediction

- You are essentially training a logistic regression classifier instead of a multi-layer network that has a lot of sophisticated algorithms and hidden layers.

5.5.2 How skip-gram works

The basic intuition is

1. Treat the target and neighboring words as a positive example.
2. Randomly sample other words to get negative samples.
3. Use logistic regression to train a classifer to distinguish between those two cases.
4. Use the learned weights as embeddings.

It trains a probabilistic classifier that assigns a probability of how similar the context is to the target word.[1]

5.5.3 Skip-gram embeddings

The learning algorithm for skip-gram embeddings is as follows

1. Take a corpus of text as input and a chosen vocabulary size N vocabulary words.
2. Assign a random embedding vector for each N vocabulary words.
3. Iteratively shift embeddings of each word w to be more like the embeddings of the words that occur nearby.

For a binary classifier we also need negative samples. (Skip-gram with negative sampling uses more negative than positive samples. ratio set with param k).

For each (w, c_{pos}) training instances, create k negative samples consisting of the target word w plus a 'noise word' c_{neg}.[2]. Noise words are chosen according to their weighted unigram probability $p_a(w)$.

$$P_\alpha(w) = \frac{count(w)^\alpha}{\sum_{w'} count(w')^\alpha}$$

Given a set of positive and negative training instances, and an initial set of embeddings, the goal of the learning algorithm is to adjust those embeddings to

- Maximize similarity of the target word, context word pairs (w, c_{pos}) drawn from the positive examples
- Minimize the similarity of the negative examples.

Then use stochastic gradient descent to calculate to move the weights and contexts towards the direction we want them to minimize loss.

[1] To see the worked out example of this, refer to https://web.stanford.edu/~jurafsky/slp3/5.pdf

[2] A noise word being a random word from the lexicon not constrained to the lexicon to the target word w

6 Neural Networks

- Fundamental computational tool for language processing
- Based on **McCulloch-Pitts** neuron (modern NNs no longer draw on these biological inspirations)
- Modern NNs are a network of small computational units that takes a vector of input and produces a single output.
- Modern NNs are often called **deep learning** because of their many layers.
- Takes raw inputs like tokens as inputs and learns to induce features to classify.

6.1 Units

The core of NNs is a single computational unit. The unit takes a set of real valued numbers as input, performs magic, and then produces an output.

Bias terms

Given a set of inputs $x_1, \ldots, x_n$ a unit has corresponding weights $w_1, \ldots, w_n$ and a bias term which can be represented as

$$z = b + \sum_i w_i x_i$$

We usually think of the weights and inputs as vectors, so it can be represented as the dot product plus the bias term

$$z = w \cdot x + b$$

Finally, we do the same thing we did in logistic regression, which is take the z the output and apply a non-linear function f to z. The output of this function is referred to as the **activation** value of the unit a. The activation for the node is the final output of the network which is called y so y is defined as

$$y = a = f(z)$$

Common activation function used are **tanh**

$$y = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

or the **ReLU** function which is just

$$y = \text{ReLU}(z) = max(z, 0)$$

6.2 XOR problem

Consider the task of computing basic logical functions of two inputs AND, OR, XOR. Here are the truth tables for each

AND			OR			XOR		
x_0	x_1	y	x_0	x_1	y	x_0	x_1	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

It's very easy to build a perceptron that can compute AND and OR, but not XOR. The problem with the perceptron is it's a linear classifier and XOR is a non-linear problem. So, what's the solution?

6.2.1 Neural networks!

The XOR function cannot be computed with a single perceptron, but it can be calculated with a layered neural network![3].

6.3 Feed-forward neural networks

A feed-forward neural network is a multi-layer neural network in which layers are connected with no cycle. Simple feed-forward neural networks have three kinds of nodes: input units, hidden units, and output units.

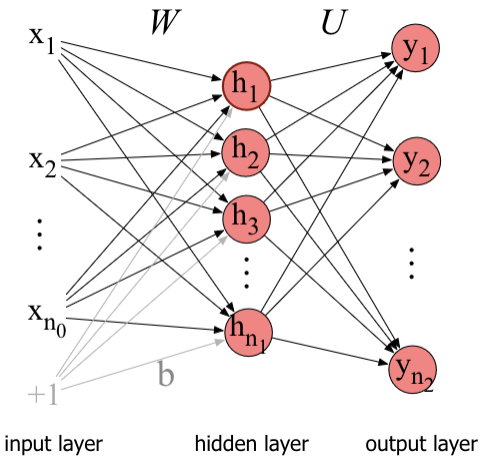


Figure 1: Example of a feed-forward neural network

A single hidden unit has parameters, a weight vector, and a bias. The parameters can be represented by combining weight vector and the bias for every input i into a single Weight metrix W and single bias vector b for the entire layer. Now we can just do matrix multiplication! GPUs are great at that!

The computation only has three steps now:

[3]See https://web.stanford.edu/~jurafsky/slp3/6.pdf#figure.6.6

1. Multiply the weight matrix by input vector **x**.
2. Add bias **b**.
3. Apply the activation function g (sigmoid, ReLU, etc.)

The output of the hidden layer would be the following

$$h = \sigma(Wx + b)$$

For the case of classification, since the output layer could contain more than one node, this can also be represented as a vector of values that gives a probability distribution across all output nodes.

6.4 Feed-forward networks for NLP classification

Now that we have the basics of neural networks down, how can we model

$$P(w_t|w_1^{t-1}) \approx P(w_t|w_{t-N+1}^{t-1})$$

We need to represent these tokens as vectors so they can be put into our neural model. The naive option would be to map each token to an arbitrary number, or one-hot encode them. This is problematic because we aren't encoding any information with this approach.

A better approach would be to use **word embeddings**. Dense representations can encode some meaning to where the words appear in a vector. It also allows the NN to generalize better with words that it has never seen before.

6.4.1 How it works in practice

First, we take a vocabulary using our training data. Each token is assigned an index and corresponding 1-hot vector. This vector is multiplied with a dense embedding layer initialized from the word embeddings, and acts as a "lookup" in our embedding matrix.

In the figure below, we can see the language modeling task that has a context of only 3 embedding vectors. Those three vectors are then concatenated to produce **e** the embedding layer. It is then multiplied by the weight matrix **W** the polygon to produce the hidden layer and another weight matrix **U** to produce an output layer whose softmax gives us the probability distribution of the words.

The equation for this simple NN looks like

$$\begin{aligned} e &= [\mathbf{E}\mathbf{x}_{t-3}; \mathbf{E}\mathbf{x}_{t-2}; \mathbf{E}\mathbf{x}_{t-1}] \\ h &= \sigma(\mathbf{W}\mathbf{e} + \mathbf{b}) \\ z &= \mathbf{U}\mathbf{h} \\ \hat{y} &= \text{softmax}(\mathbf{z}) \end{aligned}$$

The semicolons represent the concatenation of the three embeddings.

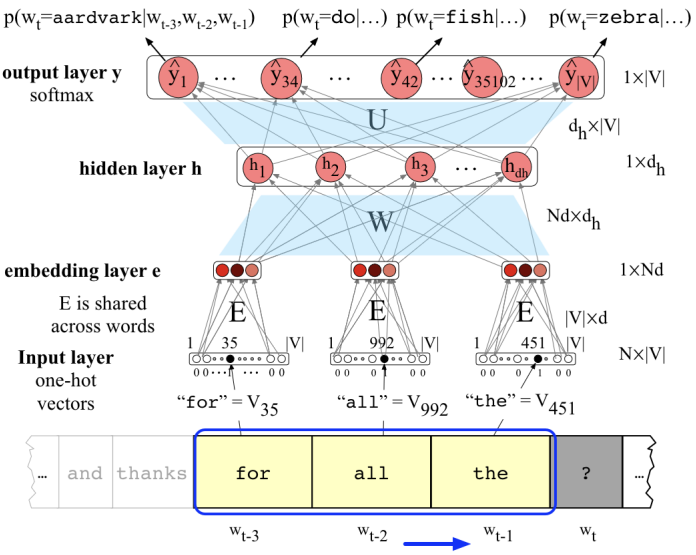


Figure 2: Forward interference in a feed-forward neural language model.

6.5 Training NNs

The goal of training a neural network is to get it to blindly throw darts very accurately. A more formal way of putting it is: to get it to learn parameters W^i and b^i for each layer i such that $\hat{y}$ is as true to y as possible.

First, we need a **loss function** (common to use cross-entropy loss). Second, we need to find parameters that minimize loss using **gradient descent** . Third, we need to know the gradient of the loss function. For this we will use **error back propagation**.

6.6 Loss function

Same thing as logistic regression

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$$

6.7 Gradient

We need the partial derivative of the loss function with respect to each parameter. For deep NNs computing gradients is much more complex as we need the derivative with respect to weight parameters that appear in every layer of the network, and not just the one at the very end. To do this, we need to use **back propagation**.

6.8 Computation graphs

A computation graph is a representation of the process of computing a mathematical expression. The expression is broken down into separate operations modeled as a node on a graph.

6.8.1 Backwards pass

This is used to compute the derivatives needed for weight updates. We can make use of the **chain rule** $f(x) = u(v(x))$:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

We can extend that to more than two functions like so

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

For the function $L(a, b, c) = c(a + 2b)$ the computation graph would look something like this

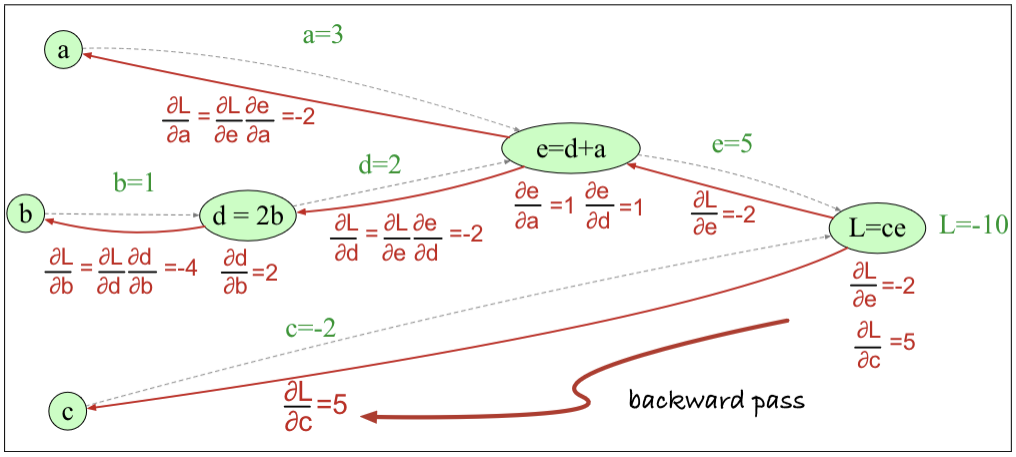


Figure 3: Computation graph of $L(a, b, c) = c(a + 2b)$

Of course a computation for a neural network is much more complicated and honestly is not worth computing by hand. Just let the damn computer do it.

6.9 Recurrent NNs

Any network that contains a cycle within its network connections. Recurrent neural networks work the same as any neural network: An input vector x_i goes in, multiplied by a weight matrix, passed through a non-linear activation function to compute values for the hidden layer. This hidden layer is then used to calculate a corresponding output y_i.

7 Transformers

Transformers are the standard architecture for building LLMs. The first types of transformers are left-to-right transformers (called causal or auto-regressive decoders) used for language modeling. These transformers work by taking an input token and predicting output tokens one by one by conditioning on the prior context.

There are three types of transformer architectures:

1. Causal autoregressive models

2. Encoder-decoder
3. Bidirectional encoders

7.1 Attention

The core foundation of transformers, and what differentiates them from neural networks, is this concept called **attention**. Attention is a "biologically inspired" concept that essentially allows you to allocate resources to focus on what's really important from a larger input field.

7.1.1 Word embeddings vs contextual embeddings

Before, we used **word embeddings** to represent a single word as a vector. But now we need something more robust.

Words (vectors) must have information from the surrounding context to make sense of that word, how to use it, etc.

7.1.2 Self-attention

Casual sequence processing (Left-to-right processing)

This works by essentially comparing the current token to each previous item in a sentence and generating a similarity score to each, normalizing these scores to produce a probability distribution, and then using the normalized scores to produce an output based on the weighted sum of the preceding units.

A simplified version

Attention is just a weighted sum of context vectors, with a lot of headaches as to how the weights are computed and what gets summed.

A simplified version of attention just boils down to

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j$$

where a_i is the attention token at position i, x_i is the weighted sum of all representations, and each α_{ij} is a scalar used for weighing the value of input x_j.

How do we compute this α weighting? We weigh each prior embedding proportionally to how **similar** it is to the current token i.

This is the attention-based approach to computing: compare the x_i prior vectors, normalize those scores in a probability distribution.

7.1.3 Attention head

This version is what's actually used in transformers. The attention head allows us to distinctly represent three different roles that each input embedding plays during the attention process:

1. **query** — the current element being compared to preceding inputs

2. **key** — a preceding input that is being compared to the current element to determine the similarity weight.
3. **value** — this gets weighted and summed to compute the output for the current element

To use these, we introduce weight matrices for each item listed above. These weights will project each input vector x_i into a representation of its role as a query, key, or value:

$$q_i = x_i W^Q \; k_i = x_i W^K \; v_i = x_i W^V$$

For a single attention head with one layer this is what the calculations would look like

$$\begin{aligned} q_i = x_i W^Q \; k_i &= x_i W^K \; v_i = x_i W^V \\ \text{score}(x_i, x_j) &= \frac{q_i \cdot k_j}{\sqrt{d_k}} \\ \alpha_{ij} &= \text{softmax}(\text{score}(x_i, x_j)) \forall j \leq i \\ \mathbf{head}_i &= \sum_{j \leq i} \alpha_{ij} v_j \\ a_i &= \mathbf{head} \; W^O \end{aligned}$$

The matrix $\mathbf{W^O}$ is used to reshape the output of the head (to make things less of a headache).

7.1.4 Multi-head attention

The above example was of a single attention head. Transformers use multiple of those each with their own set of parameters, key, query, value weight matrices, etc. Modifying the single-head attention for multi-head is almost trivial

$$\begin{aligned} q_i^c = x_i W^{Qc} \; k_i &= x_i W^{Kc} \; v_i = x_i W^{Vc} \; \forall c \; 1 \leq c \leq A \\ \text{score}^c(x_i, x_j) &= \frac{q_i^c \cdot k_j^c}{\sqrt{d_k}} \\ \alpha_{ij}^c &= \text{softmax}(\text{score}^c(x_i, x_j)) \forall j \leq i \\ \mathbf{head}_i &= \sum_{j \leq i} \alpha_{ij}^c v_j^c \\ a_i &= (\mathbf{head}^1 \oplus \mathbf{head}^2 \oplus \ldots \mathbf{head}^A) \; W^O \\ \mathbf{MultiHeadAttention}(x_i, [x_1, x_2, \ldots, x_{i-1}]) &= a_i \end{aligned}$$

MultiHeadAttention is a function of the current input x_i as well as other inputs. We still have to do reshaping using a linear projection $\mathbf{W^O} \in \mathbb{R}^{Ad_v \times d}$ resulting in vector a_i with the correct dimensions shape of $[1 \times d]$ at each input i. The dimensionality d_v is set so that $d_v = d/A$ meaning that multihead attention costs about the same in parameters as having a single head.

7.2 Transformer blocks

A transformer block is made up of these parts

1. Self-attention layer
2. Feed-forward layer
3. Residual connections
4. Normalizing layers (layer norms)

The figure below depicts how these parts integrate into the **residual stream** which is is the processing of an individual token thorugh the transformer block as a single stream of d-dimensional representations for token position i.

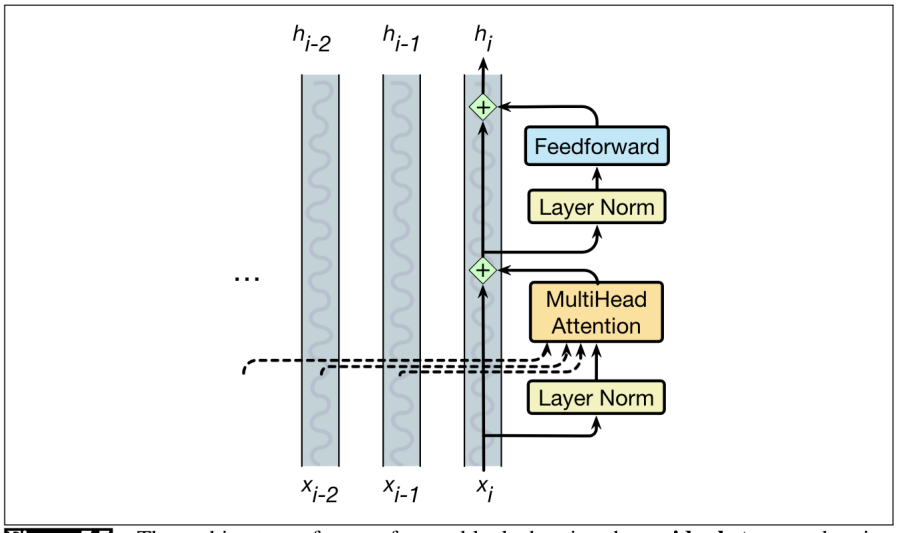


Figure 7.7 The architecture of a transformer block showing the **residual stream**, showing how most information flows up through the residual stream, and only the attention module is sensitive to information from other streams at prior token positions. In this figure and throughout the chapter, we use the **prenorm** version of the architecture, in which the layer norms happen before the attention and feedforward layers rather than after.

7.2.1 Feed-forward layer

1. The feed-forward layer is a fully-connected 2-layer network (one hidden layer, two weight matrices).
2. The feed-forward layer operates on each position i independently.
3. The FF weights are shared across positions. Same parameters applied to every token position but are different from layer to layer.

7.2.2 Layer norm

At two stages we normalize the vector using **layer normalization** (layer norm for short) to improve training performance in deep NNs by keeping the values of a hidden layer in a range that facilitates gradient-based training.

Layer norm is actually a variation of **z-score** in statistics, applied to a single vector in a hidden layer.

IMPORTANT — the layer norm is **not applied** to an entire transformer layer, but rather the embedding vector of a single token.

First, we calculate the mean μ and the standard deviation σ over the elements of the vector to be normalized.

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x[j]$$
$$\sigma = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(x[j]-\mu)^2}$$

The result is a new vector with zero mean and a standard deviation of one.

$$\hat{x} = \frac{(x-\mu)}{\sigma}$$

Finally, two learnable parameters γ and β are introduced

$$\text{LayerNorm}(x) = \gamma\frac{(x-\mu)}{\sigma} + \beta$$

7.2.3 All together now

All together the steps are as follows

$$\begin{aligned}
t_i^1 &= \text{LayerNorm}(x_i) \\
t_i^2 &= \text{MultiHeadAttention}(t_i^1, [t_i^1, \ldots, t_{i-1}^1]) \\
t_i^3 &= t_i^2 + x_i \\
t_i^4 &= \text{LayerNorm}(t_i^3) \\
t_i^5 &= \text{FFN}(t_i^4) \\
h_i &= t_i^5 + t_i^3
\end{aligned}$$